

Relatório Técnico

Teste Funcional e Implementação do Programa Identifier

1. Objetivo

Este relatório documenta a aplicação do critério funcional de Particionamento em Classes de Equivalência sobre a especificação do programa Identifier, a derivação dos casos de teste correspondentes, a complementação desses casos com Análise de Valor Limite e a implementação da execução automatizada dos testes utilizando Java e o framework JUnit 5.

Especificação validada: um identificador é válido se, e somente se: (i) começa com uma letra; (ii) contém apenas letras ou dígitos; (iii) possui comprimento entre 1 e 6 caracteres.

Passo 1: Identificação das Classes de Equivalência

Foram identificadas três dimensões independentes de entrada — comprimento, caractere inicial e tipos de caracteres contidos — cada uma particionada em uma classe válida e uma ou mais classes inválidas, conforme a tabela a seguir.

Classe	Dimensão	Tipo	Descrição	Exemplo
CE1	Comprimento	Válida	Comprimento entre 1 e 6 caracteres	model
CE2	Comprimento	Inválida	Comprimento igual a 0 (string vazia)	""
CE3	Comprimento	Inválida	Comprimento maior que 6 caracteres	abcdefg
CE4	Caractere inicial	Válida	Primeiro caractere é uma letra (a-z, A-Z)	model
CE5	Caractere inicial	Inválida	Primeiro caractere não é uma letra (dígito, espaço ou símbolo)	100tidez
CE6	Caracteres restantes	Válida	Todos os demais caracteres são letras ou dígitos	p2p
CE7	Caracteres restantes	Inválida	Algum caractere restante não é letra nem dígito (espaço, hífen, underscore, símbolo)	abc-12

Passo 2: Casos de Teste Derivados das Classes de Equivalência

Para cada classe identificada no passo 1, foi selecionado ao menos um elemento representativo, gerando os casos de teste abaixo. Esta é a mesma lista de casos já presente na planilha de controle de testes elaborada na aula 06 ATVD-04; ela foi mantida integralmente para fins de comparação e reaproveitada como base, pois cobre de forma consistente todas as classes válidas e inválidas definidas no Passo 1.

Caso	Descrição	Entrada	Classes cobertas	Resultado esperado
TC1	Identificador com 1 letra	S	CE1, CE4, CE6	Válido
TC2	Somente letras	model	CE1, CE4, CE6	Válido
TC3	Letras e dígitos	p2p	CE1, CE4, CE6	Válido
TC4	Identificador com 6 caracteres	abcdef	CE1, CE4, CE6	Válido

Caso	Descrição	Entrada	Classes cobertas	Resultado esperado
TC5	Identificador com 7 caracteres	abcdefg	CE3	Inválido
TC6	Identificador vazio	(vazio)	CE2	Inválido
TC7	Inicia com dígito	100tidez	CE5	Inválido
TC8	Contém símbolo especial	Te@st	CE7	Inválido
TC9	Contém espaço (interno)	abc def	CE7	Inválido
TC10	Começa com espaço	" 6seven"	CE5	Inválido
TC11	Muito grande	stringmuitogrande	CE3	Inválido
TC12	Contém hífen	abc-12	CE7	Inválido
TC13	Contém underscore	abc_12	CE7	Inválido
TC14	Letras maiúsculas	ABC123	CE1, CE4, CE6	Válido
TC15	6 caracteres válidos	a12345	CE1, CE4, CE6	Válido
TC16	Formado somente por números	123456	CE5	Inválido

2. Adicional — Análise de Valor Limite

A Análise de Valor Limite complementa o particionamento ao testar deliberadamente os valores nos extremos e imediatamente fora dos extremos de cada classe — nesse caso, os limites são 1 e 6 caracteres.

Tamanho	Exemplo	Posição em relação ao limite	Classe (Passo 1)	Resultado esperado	Caso de teste
0	(vazio)	Um a menos que o mínimo válido	CE2	Inválido	TC6
1	s	Exatamente no limite inferior válido	CE1	Válido	TC1
6	abcdef	Exatamente no limite superior válido	CE1	Válido	TC4
7	abcdefg	Um a mais que o máximo válido	CE3	Inválido	TC5

Os quatro valores de fronteira (0, 1, 6 e 7) já estão cobertos pelos casos TC6, TC1, TC4 e TC5, respectivamente. O particionamento em classes de equivalência elaborado na aula anterior, por escolher elementos exatamente nos extremos das classes CE1, CE2 e CE3, já incorporava a análise de valor limite.

5. Implementação do Programa e dos Testes

O algoritmo do programa Identifier foi implementado em Java na classe `Identifier.java` (pacote `testIdentifier`), com o método público `validateIdentifier(String s)`, que retorna `true` quando a entrada satisfaz simultaneamente as três regras da especificação, e `false` caso contrário. Uma classe auxiliar `IdentifierMain.java` expõe o programa via linha de comando.

A execução dos testes foi implementada em JUnit 5, na classe `IdentifierTest.java`. Cada caso de teste corresponde a um método de teste e segue explicitamente a estrutura formal exigida:

- **Setup:** inicialização do objeto `Identifier` (via `@BeforeEach`) e definição da entrada do caso de teste.
- **Invocation:** chamada do método `validateIdentifier(entrada)` com o dado de teste.

- **Assessment (Asserts):** comparação do resultado obtido com o resultado esperado, por meio de `assertTrue / assertFalse`.

Ao total, a suíte contém 17 métodos de teste, os 16 casos da tabela (TC1–TC16) mais um teste explícito de valor limite (comprimento igual a 0), incluído para evidenciar separadamente a análise do item 2 deste relatório.

6. Execução dos Testes — Resumo dos Resultados

A suíte foi executada com o JUnit Platform Console Launcher. Todos os testes foram executados com sucesso, sem falhas:

Métrica	Valor
Testes encontrados	17
Testes executados	17
Testes com sucesso (pass)	17
Testes com falha (fail)	0
Testes ignorados (skipped)	0

O resultado confirma que a implementação de `Identifier.java` atende corretamente à especificação para todas as classes de equivalência válidas e inválidas identificadas no Passo 1, incluindo os quatro valores de fronteira analisados no item 4 (0, 1, 6 e 7 caracteres).

7. Conclusão

A aplicação do Particionamento em Classes de Equivalência permitiu derivar um conjunto enxuto e sistemático de 16 casos de teste, cobrindo tanto as classes válidas quanto as inválidas de cada dimensão da especificação do programa `Identifier`. A Análise de Valor Limite, aplicada sobre a dimensão de comprimento, confirmou que esses mesmos casos já cobriam as fronteiras mais propensas a defeitos (0, 1, 6 e 7 caracteres). A automação em JUnit 5, estruturada em Setup / Invocation / Assessment, validou a totalidade dos casos com sucesso.